

Семинар по C++ №14

Шаблоны. Часть 1

Мотивация использования шаблонов

- Не хотим дублировать один и тот же код для разных типов
- Примеры:
 - max, swap, ...

```
int max(int a, int b) {  
    return a > b ? a : b;  
}
```

```
float max(float a, float b) {  
    return a > b ? a : b;  
}
```

Простейший пример использования

```
template <typename T>  
T max(T a, T b) {  
    return a > b ? a : b;  
}
```

```
max(1.0, 2.0);  
max(1, 2);
```

Теперь max это не функция,
а шаблон функции.
При использовании
функции подставляется
необходимый тип и
генерируется код функции —
происходит
**инстанцирование
шаблона.**

Шаблонные классы

```
template <typename T>
class Test {
    T obj;
    T* obj_ptr;
    std::vector<T> vec;
    void foo(T x);
    ...
};
```

Можно создавать шаблонные классы и работать в них с шаблонной переменной как с обычным типом. Точно так же реальный класс будет инстанцирован с реальным типом компилятором

Шаблонные using и переменные

```
template <typename T>
```

```
using MyPair = std::pair<std::unordered_map<T, T>,  
std::map<std::vector<std::string>, std::.....>>>;
```

```
template <typename T>
```

```
T template_var;
```

```
MyPair<int> pair;
```

```
template_var<double> var;
```

Сигнатура функции

- В отличие от обычных функций, возвращаемое значение шаблонных функций входит в сигнатуру, поэтому не является СЕ следующее:

```
template<class T> int foo(T) {}
```

```
template<class T> bool foo(T) {}
```

Двухэтапная компиляция шаблонов

1. Сначала проверка шаблона на корректность (максимально возможная проверка без знания шаблонных параметров)
2. Проверка инстанцированного шаблона

```
template <typename T>
int foo(T x) {
    return std::vector<int>();
}
```

```
template <typename T>
int foo(T x) {
    return x;
}

int main() {
    foo(std::vector<int>());
}
```

Template argument deduction (вывод шаблонных аргументов)

```
template <typename T>
```

```
T max(T a, T b);
```

max(1, 0) - **OK**

max(1.0, 2.0) - **OK**

max(1, 2.0) – **CE**

template argument deduction/substitution failed: deduced conflicting types for parameter 'T'

Правила вывода шаблонных аргументов

- Компиляция заканчивается ошибкой при:
 1. Неоднозначно, какой тип выбрать в качестве шаблонного параметра (при выборе типа касты не происходят)
 2. Не является аргументом функции
 3. Когда аргумент имеет “сложный” тип (e.g. `std::vector<T>::iterator`)

Можно явно указать, с каким шаблонным параметром нужен шаблон:

```
max<float>(1, 1.0)
```

Type deduction

```
template <typename T>  
void f(ParamType param);  
f(expr);
```

Типы ParamType и тип expr
могут не совпадать

Если ParamType является указателем или ссылкой:

1. Если тип expr это ссылка, игнорируем ссылочность
2. Паттерн-матчинг типа expr в ParamType для выяснения T

Если ParamType **не** является ни указателем, ни ссылкой:

1. Если тип expr это ссылка, игнорируем ссылочность (как и раньше)
2. Игнорируем константность тоже (так как всё равно создаётся копия объекта)

Type deduction

- При передаче массива в функцию, принимающую T, происходит decay массива до указателя
- Однако, если функция принимает T&, то тип T будет массив соответствующего массива
- Ровно то же самое происходит и с функциями: либо decay до указателя, либо ссылка на функцию

Перегрузка шаблонных функций (overload)

1. **Частное предпочтительнее общего** (общее – шаблонный параметр, частное – функция, с конкретным типом)
2. **Точное соответствие лучше приведения типа** (если есть точное соответствие типов (даже в более общей версии), то это лучше чем каст)

Специализация шаблонов классов

- Можно указать, что реализация класса с некоторым шаблонным параметром отличается от остальных:

```
template <typename T, typename U>  
class Test {  
    ...  
};
```

```
template <>  
class Test<int, double> {  
    ...  
};
```

Полная специализация

```
template <typename T, typename U>  
class Test {  
    ...  
};
```

```
template <typename T>  
class Test<T, std::vector<T>> {  
    ...  
};
```

Частичная специализация

Специализация шаблонов функции

Запрещена частичная специализация, есть только полная (она не нужна, есть перегрузка). То есть, в специализации **всегда** `template <>`

```
template <typename U, typename V>  
void foo(U a, V b) {}
```

```
template <>  
void foo(int a, double b) {}
```

Специализация шаблонов функций

- Специализация всегда цепляет/специализирует ближайший подходящий шаблон, который идёт до неё по коду:

```
template <typename U, typename V>  
void foo(U a, V b) {}
```

```
template <typename U>  
void foo(U a, U b) {}
```

```
template <>  
void foo(int a, int b) {}
```

Специализация второго шаблона

```
template <>  
void foo(int a, double b) {}
```

Специализация первого шаблона

Правила перегрузок при специализации

1. Сначала выбирается наиболее подходящая нешаблонная функция или шаблон
2. Затем выбирается специализация шаблона

Integral template parameter

Шаблонным параметром может быть не только тип, но и целое число

```
template <size_t M, size_t N>  
class Matrix {  
    ...  
};
```